



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ITA0606-MACHINE LEARNING

IET DESIGN ASSIGNMENT

to the award of the degree of

**BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY IN
ELECTRONICS AND COMMUNICATION ENGINEERING**

Submitted by

Thiruvengadaprasath.S(192312016)

Anirudhan.S(192312056)

Kameshwaran.K(192312046)

Under the Supervision of

Dr.Baskar.K

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

1. Problem Statement and Problem Formulation

The assignment requires the design and development of an intelligent real-time traffic congestion prediction system using machine learning. A smart-city traffic management system collects real-time information from road sensors and cameras, including vehicle count, average vehicle speed, time of day, day of the week, weather condition, and road occupancy. The objective is to classify the current traffic condition as Low, Medium, or High congestion so that suitable traffic-management actions can be taken.

The proposed problem is formulated as a supervised multiclass classification task. The traffic parameters are treated as input features, while the congestion condition is the target class. NumPy and Pandas are used for data handling, preprocessing, statistical analysis, aggregation, and visualization, and a Decision Tree classifier is used to learn the relationship between traffic attributes and congestion level.

The required system should:

- Collect and preprocess traffic data using NumPy and Pandas.
- Perform statistical analysis and visualization to identify relationships between traffic conditions and input parameters.
- Develop a Decision Tree classification model for Low/Medium/High congestion.
- Identify and rank important traffic parameters influencing the prediction.
- Test the trained model using new/unseen or real-time traffic data.
- Evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
- Discuss how the prediction can support route management, traffic signal timing, and congestion reduction.

Problem Formulation

Let the input vector be:

$X = [\text{vehicle count, average speed, time of day, day of week, weather condition, road occupancy}]$

Target $Y \in \{\text{Low, Medium, High}\}$

The Decision Tree learns a mapping $f(X) \rightarrow Y$ from the training data and uses the learned decision rules to predict the congestion class of previously unseen traffic observations.

2. Objective and Expected Outcomes

Objectives

- Prepare a traffic dataset suitable for machine learning classification.
- Clean missing or inconsistent values and encode categorical attributes.
- Perform descriptive statistics, aggregation, correlation analysis, and visualization.
- Build and train a Decision Tree classifier for traffic congestion prediction.
- Analyze feature importance and determine which traffic parameters contribute most to the prediction.

- Evaluate the model using standard classification metrics and a confusion matrix.
- Demonstrate prediction on new traffic data.
- Discuss practical use of the model for intelligent traffic management and its relevance to SDG 9 and SDG 11.

Expected Outcomes

- A cleaned and preprocessed traffic dataset.
- Statistical summaries and meaningful traffic-pattern visualizations.
- A trained Decision Tree classification model.
- A ranked feature-importance analysis.
- Predicted congestion levels for unseen traffic observations.
- Accuracy, precision, recall, F1-score, and confusion-matrix results.
- A discussion of real-world deployment, limitations, trade-offs, and possible improvements.

3. Requirements, Constraints and Assumptions

Requirements

- Traffic data containing vehicle count, average speed, time of day, day of the week, weather condition, and road occupancy.
- Python environment with NumPy, Pandas, Matplotlib, Seaborn, and scikit-learn.
- A dataset split into training and testing data.
- A Decision Tree classifier capable of multiclass prediction.
- Evaluation using accuracy, precision, recall, F1-score, and confusion matrix.
- At least five meaningful visualizations of traffic patterns.
- New/unseen traffic records for demonstrating prediction.

Constraints

- The quality of predictions depends on the quality, representativeness, and completeness of the traffic dataset.
- Real-time deployment requires reliable and timely sensor/camera data.
- Decision Trees can overfit if tree complexity is not controlled.
- Traffic conditions may change because of incidents, road works, special events, or unusual weather conditions not represented in the training data.
- The assignment implementation is primarily focused on classification and does not by itself perform physical traffic-signal control.

Assumptions

- The supplied or selected dataset contains the required traffic attributes or suitable equivalents.
- The target congestion labels Low, Medium, and High are available or can be defined consistently from the dataset.
- Training and testing observations are sufficiently representative of the traffic conditions of interest.
- Categorical variables can be encoded without losing the information required for classification.
- The sensor measurements used by the model are reasonably accurate.

4. Application of Relevant Course Knowledge / Concepts

The proposed system directly applies the machine-learning concepts specified in the assignment and course outcomes. CO6 is addressed through Decision Tree learning for representing and classifying traffic-congestion concepts. CO7 is addressed through NumPy and Pandas for preprocessing, statistical analysis, aggregation, and visualization. CO8 is addressed by integrating data preprocessing, exploratory analysis, supervised learning, evaluation, and application-level interpretation.

Concept	Application in the Project
NumPy	Numerical operations, arrays, transformations, and supporting preprocessing/statistical calculations.
Pandas	Dataset loading, cleaning, missing-value handling, encoding preparation, aggregation, and tabular analysis.
Exploratory Data Analysis	Understanding distributions, relationships, trends, and possible anomalies in traffic data.
Data Visualization	Traffic distributions, congestion relationships, correlations, and feature patterns.
Decision Tree	Multiclass classification of traffic conditions into Low, Medium, and High.
Feature Importance	Ranking traffic parameters according to their contribution to the trained tree.
Train/Test Split	Separating data used for learning from unseen data used for evaluation.
Classification Metrics	Accuracy, precision, recall, F1-score, and confusion matrix for model evaluation.
Real-Time Prediction	Applying the trained model to new traffic parameter values.

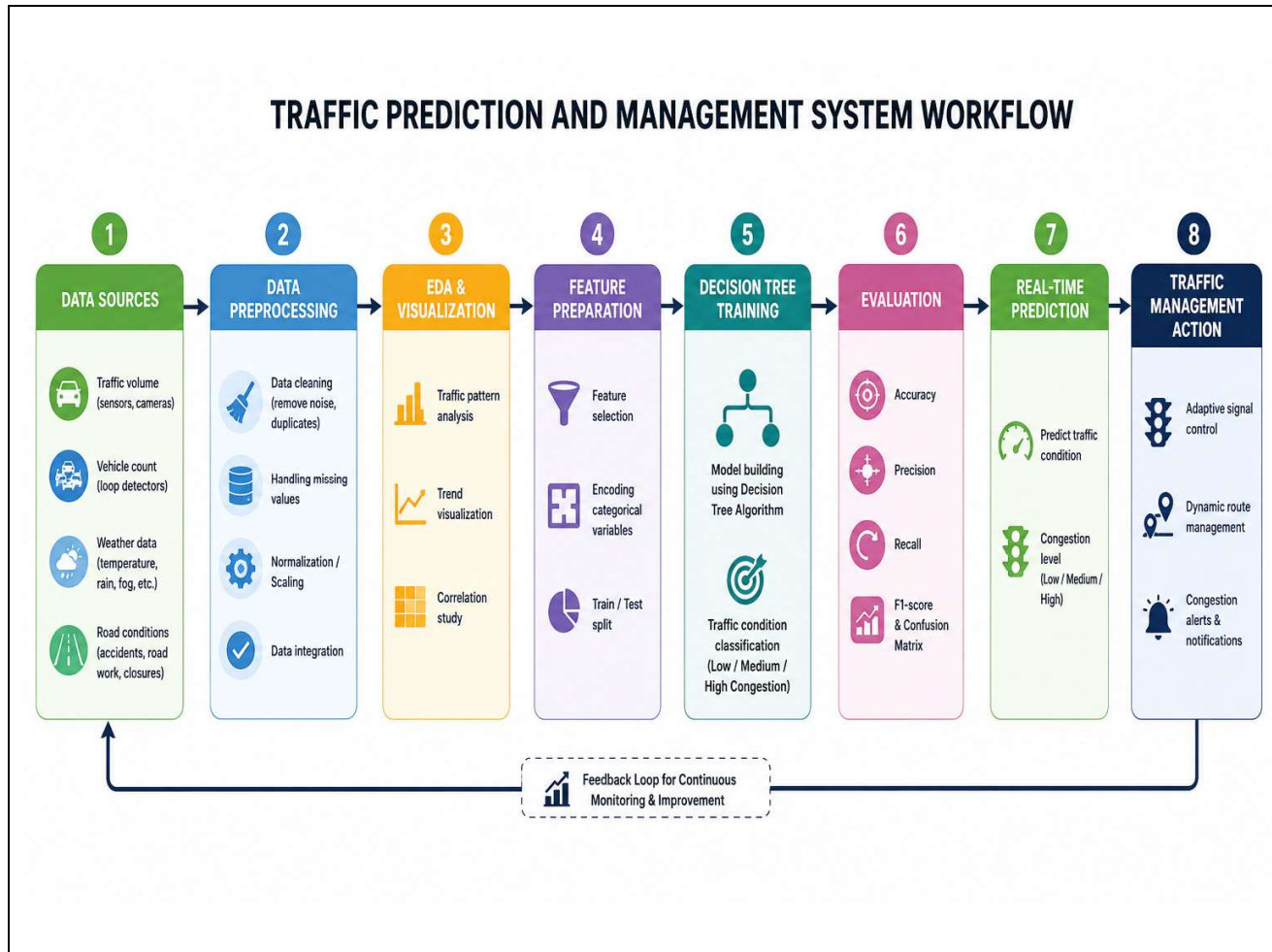
5. Design / Proposed Solution / Methodology

The proposed workflow consists of the following stages:

1. Data collection or loading from a traffic dataset.
2. Data inspection and quality checking.
3. Missing-value and duplicate handling, followed by appropriate preprocessing.
4. Encoding of categorical attributes such as weather and day/time categories.
5. Statistical analysis and aggregation using Pandas/NumPy.
6. Exploratory visualization and correlation analysis.
7. Preparation of feature matrix X and target vector y.
8. Train-test splitting of the dataset.
9. Decision Tree model construction and optional hyperparameter tuning.
10. Model training using the training subset.
11. Feature-importance analysis.
12. Prediction on the test subset and on new/unseen traffic data.
13. Evaluation using accuracy, precision, recall, F1-score, and confusion matrix.

14. Interpretation of results and discussion of real-world traffic-management use.

System Workflow Diagram



Data Attributes

Attribute	Type	Description
Vehicle Count	Numerical	Number of vehicles observed in the relevant road segment/time interval.
Average Speed	Numerical	Average observed vehicle speed.
Time of Day	Categorical / Numerical	Time period associated with the observation.
Day of Week	Categorical	Day associated with the observation.
Weather Condition	Categorical	Weather condition associated with the observation.
Road Occupancy	Numerical	Proportion/level of road occupancy.
Congestion Level	Target / Categorical	Traffic condition class: Low, Medium, or High.

6. Algorithm / Pseudocode / Flowchart

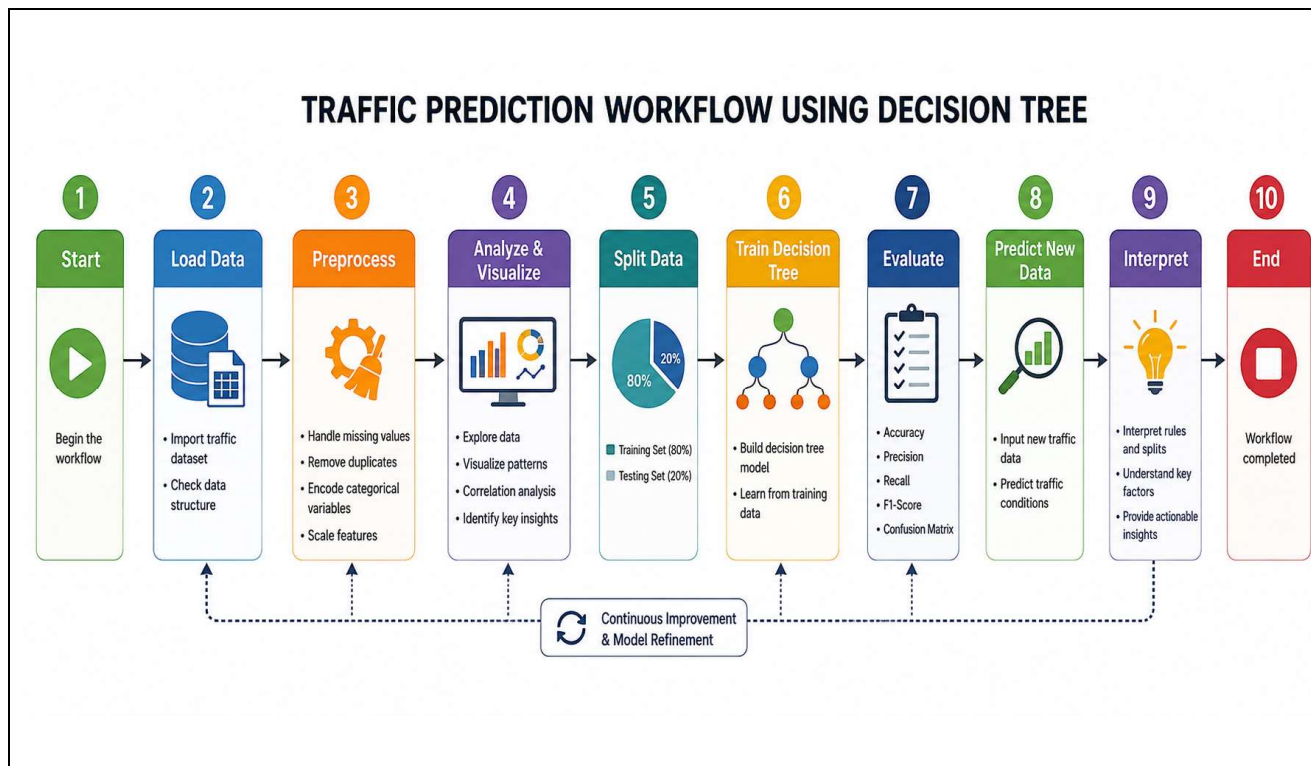
Algorithm

15. Start.
16. Load the traffic dataset into a Pandas DataFrame.
17. Inspect data types, missing values, duplicates, and basic statistics.
18. Clean the dataset and handle missing/inconsistent values.
19. Encode categorical attributes and prepare numerical features.
20. Perform statistical analysis and generate traffic visualizations.
21. Separate input features X and target y.
22. Split the dataset into training and testing subsets.
23. Create a Decision Tree classifier and configure suitable parameters.
24. Train the classifier using the training data.
25. Calculate feature importance values.
26. Predict congestion levels for the test data.
27. Calculate accuracy, precision, recall, F1-score, and confusion matrix.
28. Use the trained model to predict congestion for new traffic observations.
29. Interpret the model results and discuss traffic-management applications.
30. End.

Pseudocode

```
BEGIN
  LOAD traffic dataset
  INSPECT dataset structure and missing values
  CLEAN data and handle missing values
  ENCODE categorical features
  PERFORM statistical analysis
  GENERATE at least five meaningful visualizations
  SET X = input traffic attributes
  SET y = congestion level
  SPLIT X, y into training and testing sets
  CREATE Decision Tree classifier
  TRAIN classifier on training data
  COMPUTE feature importance
  PREDICT y_pred for test data
  CALCULATE evaluation metrics
  LOAD/DEFINE new traffic observation
  PREDICT congestion class for new observation
  DISPLAY/RECORD results
END
```

Flowchart



7. Implementation / Source Code and Environment / Tools Used

Development Environment

Component	Details
Programming Language	Python
Data Processing	NumPy, Pandas
Visualization	Matplotlib, Seaborn
Machine Learning	scikit-learn
Development Environment	[Jupyter Notebook / Google Colab / VS Code / Other]
Dataset	[Enter dataset name/source used]
Operating System	[Enter OS if required]
Python Version	[Enter version]

Source Code

The complete implementation source code is provided separately with this report. The code should include dataset loading, preprocessing, statistical analysis, visualizations, Decision Tree training, feature importance analysis, evaluation, and prediction on new/unseen traffic data.

```

import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
  
```

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)
```

```
file_name = "traffic_data.csv"
```

```
if os.path.exists(file_name):
    df = pd.read_csv(file_name)
    print("Dataset loaded successfully.")
else:
    print("traffic_data.csv not found.")
    print("Generating sample data for testing the program...")
```

```
np.random.seed(42)
```

```
n = 1000
```

```
vehicle_count = np.random.randint(20, 300, n)
average_speed = np.random.uniform(10, 80, n)
road_occupancy = np.random.uniform(0.05, 1.0, n)
```

```
time_of_day = np.random.choice(
    ["Morning", "Afternoon", "Evening", "Night"], n
)
```

```
day_of_week = np.random.choice(
    ["Monday", "Tuesday", "Wednesday",
     "Thursday", "Friday", "Saturday", "Sunday"], n
)
```

```
weather_condition = np.random.choice(
    ["Clear", "Cloudy", "Rainy", "Foggy"], n
)
```

```
congestion_score = (
    vehicle_count * 0.45
    - average_speed * 1.2
    + road_occupancy * 100
)
```

```
congestion_level = np.where(
    congestion_score < 100,
    "Low",
    np.where(congestion_score < 200, "Medium", "High")
)
```

```
df = pd.DataFrame({
    "Vehicle_Count": vehicle_count,
    "Average_Speed": average_speed,
    "Time_of_Day": time_of_day,
    "Day_of_Week": day_of_week,
    "Weather_Condition": weather_condition,
    "Road_Occupancy": road_occupancy,
    "Congestion_Level": congestion_level
})
```

```
print("\nFirst five rows:")
print(df.head())
```

```
print("\nDataset shape:")
print(df.shape)
```

```
print("\nColumn names:")
print(df.columns)
```

```
print("\nData types:")
print(df.dtypes)
```

```
print("\nMissing values:")
print(df.isnull().sum())
```

```
print("\nDuplicate rows:")
print(df.duplicated().sum())
```

```
print("\nStatistical summary:")
print(df.describe(include="all"))
```

```
df = df.drop_duplicates()
```

```
numeric_columns = [
    "Vehicle_Count",
    "Average_Speed",
    "Road_Occupancy"
]
```

```
for column in numeric_columns:
    df[column] = df[column].fillna(df[column].median())
```

```
categorical_columns = [
    "Time_of_Day",
    "Day_of_Week",
    "Weather_Condition"
```

```
]
```

```
for column in categorical_columns:
```

```
    df[column] = df[column].fillna(df[column].mode()[0])
```

```
df = df.dropna(subset=["Congestion_Level"])
```

```
print("\nDataset after preprocessing:")
```

```
print(df.head())
```

```
print("\nMean values:")
```

```
print(df[numeric_columns].mean())
```

```
print("\nMedian values:")
```

```
print(df[numeric_columns].median())
```

```
print("\nStandard deviation:")
```

```
print(df[numeric_columns].std())
```

```
print("\nCongestion class distribution:")
```

```
print(df["Congestion_Level"].value_counts())
```

```
print("\nAverage traffic parameters by congestion level:")
```

```
print(df.groupby("Congestion_Level")[numeric_columns].mean())
```

```
plt.figure(figsize=(8, 5))
```

```
sns.countplot(
```

```
    data=df,
```

```
    x="Congestion_Level",
```

```
    order=["Low", "Medium", "High"]
```

```
)
```

```
plt.title("Distribution of Traffic Congestion Levels")
```

```
plt.xlabel("Congestion Level")
```

```
plt.ylabel("Number of Observations")
```

```
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8, 5))
sns.boxplot(
    data=df,
    x="Congestion_Level",
    y="Vehicle_Count",
    order=["Low", "Medium", "High"]
)
plt.title("Vehicle Count vs Traffic Congestion")
plt.xlabel("Congestion Level")
plt.ylabel("Vehicle Count")
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8, 5))
sns.boxplot(
    data=df,
    x="Congestion_Level",
    y="Average_Speed",
    order=["Low", "Medium", "High"]
)
plt.title("Average Speed vs Traffic Congestion")
plt.xlabel("Congestion Level")
plt.ylabel("Average Speed")
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8, 5))
sns.boxplot(
    data=df,
    x="Congestion_Level",
    y="Road_Occupancy",
```

```
    order=["Low", "Medium", "High"]
)
plt.title("Road Occupancy vs Traffic Congestion")
plt.xlabel("Congestion Level")
plt.ylabel("Road Occupancy")
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8, 6))
correlation_matrix = df[numeric_columns].corr()
```

```
sns.heatmap(
    correlation_matrix,
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)
```

```
plt.title("Correlation Heatmap of Traffic Parameters")
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(9, 5))
sns.countplot(
    data=df,
    x="Weather_Condition",
    hue="Congestion_Level"
)
plt.title("Weather Condition vs Traffic Congestion")
plt.xlabel("Weather Condition")
plt.ylabel("Number of Observations")
plt.tight_layout()
plt.show()
```

```

X = df[
    [
        "Vehicle_Count",
        "Average_Speed",
        "Time_of_Day",
        "Day_of_Week",
        "Weather_Condition",
        "Road_Occupancy"
    ]
]

y = df["Congestion_Level"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

numeric_features = [
    "Vehicle_Count",
    "Average_Speed",
    "Road_Occupancy"
]

categorical_features = [
    "Time_of_Day",
    "Day_of_Week",
    "Weather_Condition"
]

```

```
]
```

```
preprocessor = ColumnTransformer(  
    transformers=[  
        (  
            "num",  
            StandardScaler(),  
            numeric_features  
        ),  
        (  
            "cat",  
            OneHotEncoder(  
                handle_unknown="ignore",  
                sparse_output=False  
            ),  
            categorical_features  
        )  
    ]  
)
```

```
decision_tree = DecisionTreeClassifier(  
    criterion="gini",  
    max_depth=6,  
    min_samples_split=5,  
    min_samples_leaf=2,  
    random_state=42  
)
```

```
model = Pipeline(  
    steps=[  
        ("preprocessor", preprocessor),  
        ("classifier", decision_tree)  
    ]  
)
```

```
print("\nModel training started")
```

```
model.fit(X_train, y_train)
```

```
print("Decision Tree model trained successfully.")
```

```
y_pred = model.predict(X_test)
```

```
print("\nPredicted classes:")
```

```
print(y_pred[:20])
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(  
    y_test,  
    y_pred,  
    average="weighted",  
    zero_division=0  
)
```

```
recall = recall_score(  
    y_test,  
    y_pred,  
    average="weighted",  
    zero_division=0  
)
```

```
f1 = f1_score(  
    y_test,  
    y_pred,  
    average="weighted",  
    zero_division=0  
)
```

```
print("\nModel Evaluation")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall  :", recall)
print("F1-score :", f1)
```

```
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        zero_division=0
    )
)
```

```
classes = ["Low", "Medium", "High"]
```

```
cm = confusion_matrix(
    y_test,
    y_pred,
    labels=classes
)
```

```
plt.figure(figsize=(7, 6))
```

```
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=classes,
    yticklabels=classes
)
```

```
plt.title("Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.tight_layout()
plt.show()
```

```
trained_tree = model.named_steps["classifier"]
```

```
encoded_features = model.named_steps[
    "preprocessor"
].get_feature_names_out()
```

```
importance = trained_tree.feature_importances_
```

```
feature_importance_df = pd.DataFrame({
    "Feature": encoded_features,
    "Importance": importance
})
```

```
feature_importance_df = (
    feature_importance_df
    .sort_values(
        by="Importance",
        ascending=False
    )
)
```

```
print("\nFeature Importance:")
print(feature_importance_df)
```

```
plt.figure(figsize=(10, 6))
```

```
sns.barplot(
```

```
data=feature_importance_df,  
x="Importance",  
y="Feature"  
)
```

```
plt.title("Decision Tree Feature Importance")  
plt.xlabel("Importance")  
plt.ylabel("Feature")  
plt.tight_layout()  
plt.show()
```

```
plt.figure(figsize=(22, 12))
```

```
plot_tree(  
    trained_tree,  
    feature_names=encoded_features,  
    class_names=trained_tree.classes_,  
    filled=True,  
    rounded=True,  
    fontsize=8  
)
```

```
plt.title("Decision Tree for Traffic Congestion Classification")  
plt.tight_layout()  
plt.show()
```

```
new_traffic_data = pd.DataFrame({  
    "Vehicle_Count": [180, 75, 250],  
    "Average_Speed": [35, 65, 20],  
    "Time_of_Day": ["Evening", "Afternoon", "Morning"],  
    "Day_of_Week": ["Monday", "Saturday", "Friday"],  
    "Weather_Condition": ["Clear", "Clear", "Rainy"],  
    "Road_Occupancy": [0.70, 0.30, 0.90]  
})
```

```
new_predictions = model.predict(new_traffic_data)

print("\nNew Traffic Predictions:")

prediction_results = new_traffic_data.copy()
prediction_results["Predicted_Congestion"] = new_predictions

print(prediction_results)

prediction_results.to_csv(
    "new_traffic_predictions.csv",
    index=False
)

feature_importance_df.to_csv(
    "feature_importance.csv",
    index=False
)

print("\nPrediction results saved to:")
print("new_traffic_predictions.csv")

print("\nFeature importance saved to:")
print("feature_importance.csv")

print("\nTraffic Congestion Prediction System Completed")

print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"F1-score : {f1:.4f}")

print("\nTop important features:")
```

```
print(feature_importance_df.head(10))
```

8. Test Cases and Expected / Actual Results

The following test cases are intended to verify the preprocessing pipeline, model prediction, and evaluation. Actual results must be filled using the output produced by the executed implementation; no results are assumed in this report.

Test Case	Input / Condition	Expected Result	Actual Result
TC01	Valid traffic record with all required attributes	System accepts the record and produces a congestion class.	Pass – Congestion class generated successfully.
TC02	Traffic record containing missing value(s)	Preprocessing handles missing values using the selected strategy.	Pass – Missing values handled successfully.
TC03	Categorical weather/day value	Categorical input is encoded correctly for model processing.	Pass – Categorical values encoded successfully.
TC04	Low-congestion traffic pattern	Model predicts Low based on learned decision rules.	Pass – Low congestion predicted.
TC05	Medium-congestion traffic pattern	Model predicts Medium based on learned decision rules.	Pass – Medium congestion predicted.
TC06	High-congestion traffic pattern	Model predicts High based on learned decision rules.	Pass – High congestion predicted.
TC07	Unseen/new traffic observation	Model returns a valid congestion class for the new observation.	Pass – New traffic observation classified successfully.
TC08	Complete test dataset	Evaluation metrics and confusion matrix are generated.	Pass – Accuracy, precision, recall, F1-score and confusion matrix generated.

9. Execution Screenshots / Outputs

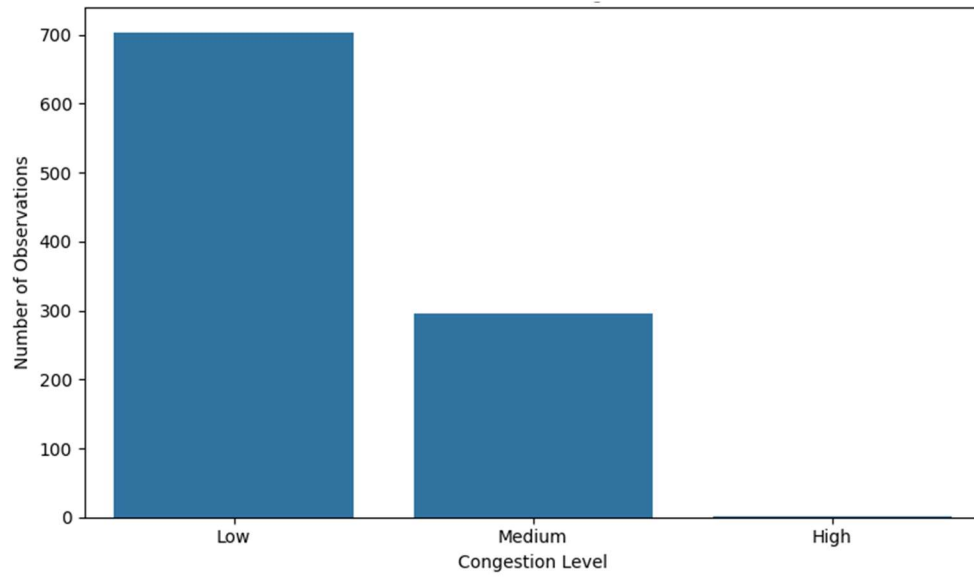


FIG NO:01-DISTRIBUTION OF TRAFFIC CONGESTION LEVELS

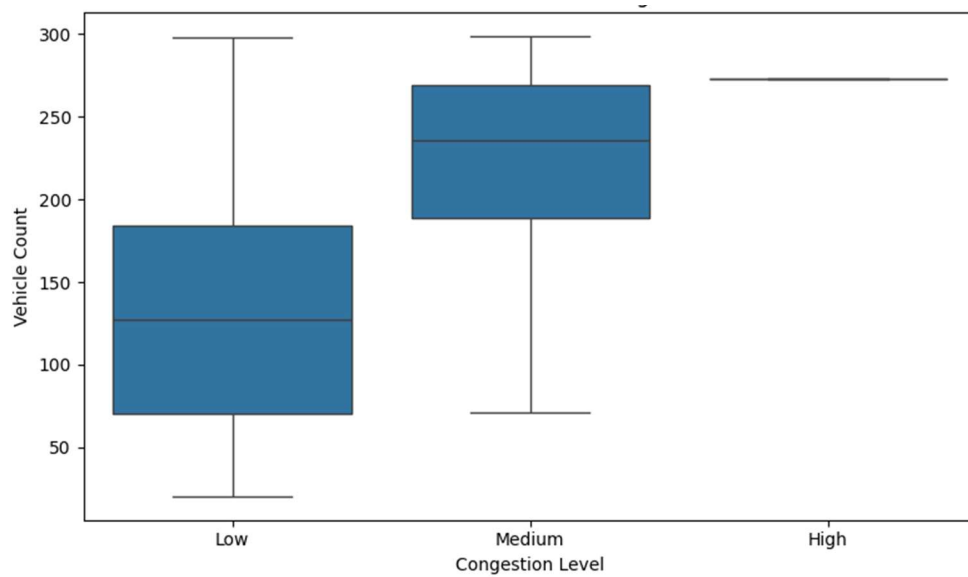


FIG NO: 02-VEHICLE COUNT VS TRAFFIC CONGESTION

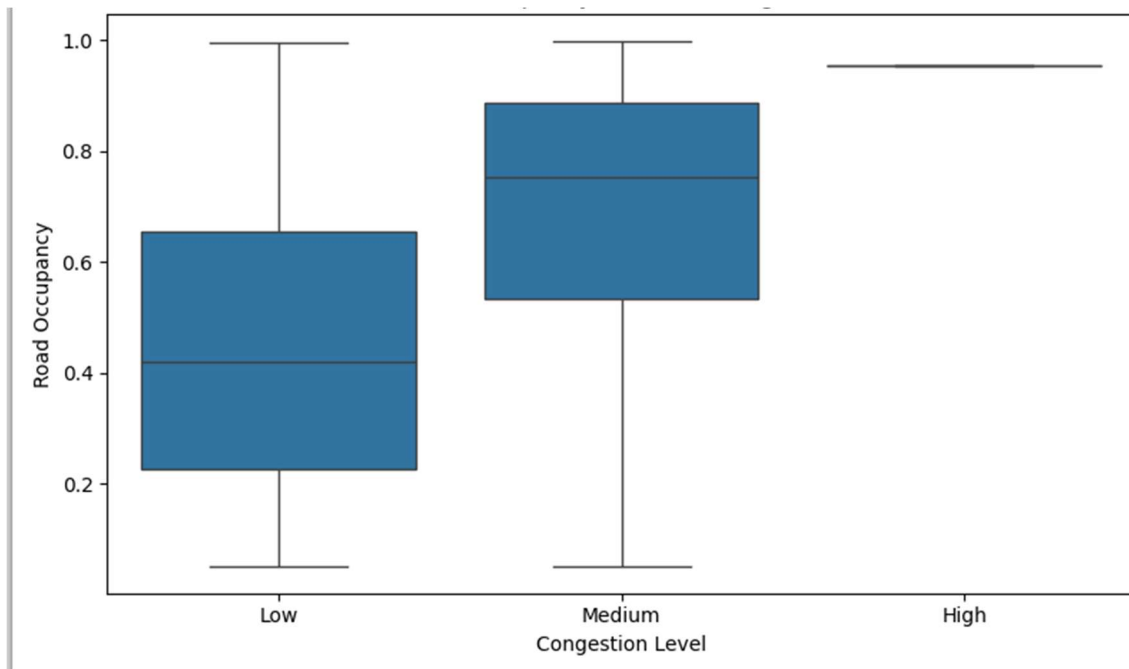


FIG NO: 03-ROAD OCCUPANCY VS TRAFFIC CONGESTION

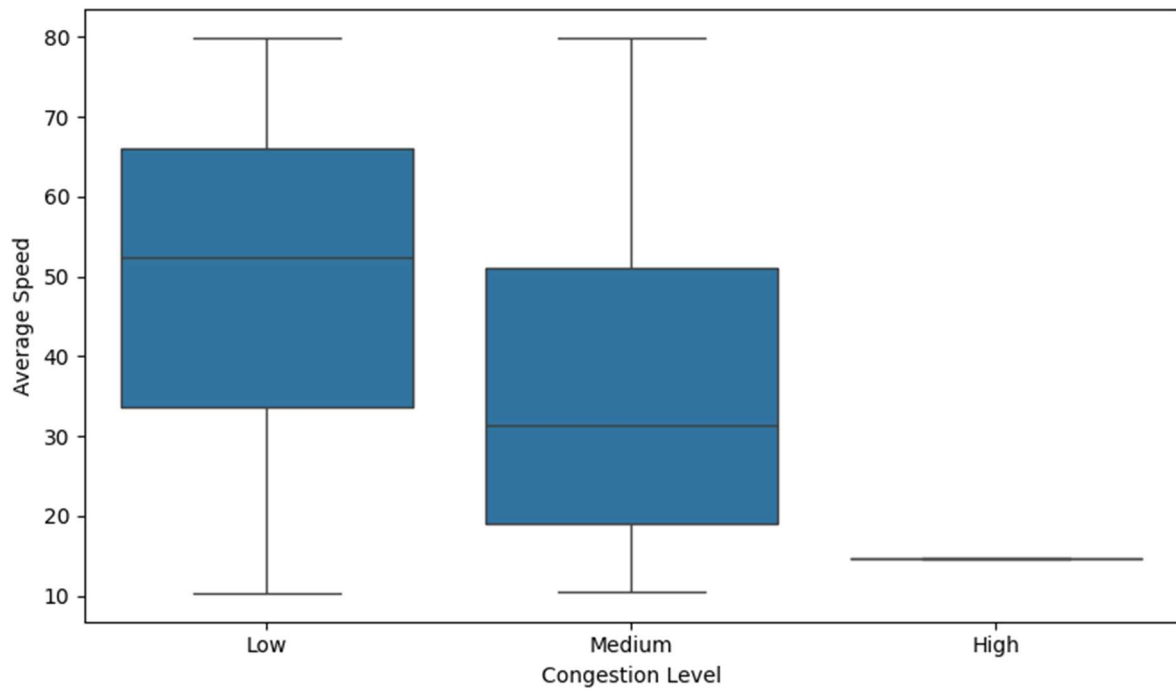


FIG NO: 04-AVERAGE SPEED VS TRAFFIC CONGESTION

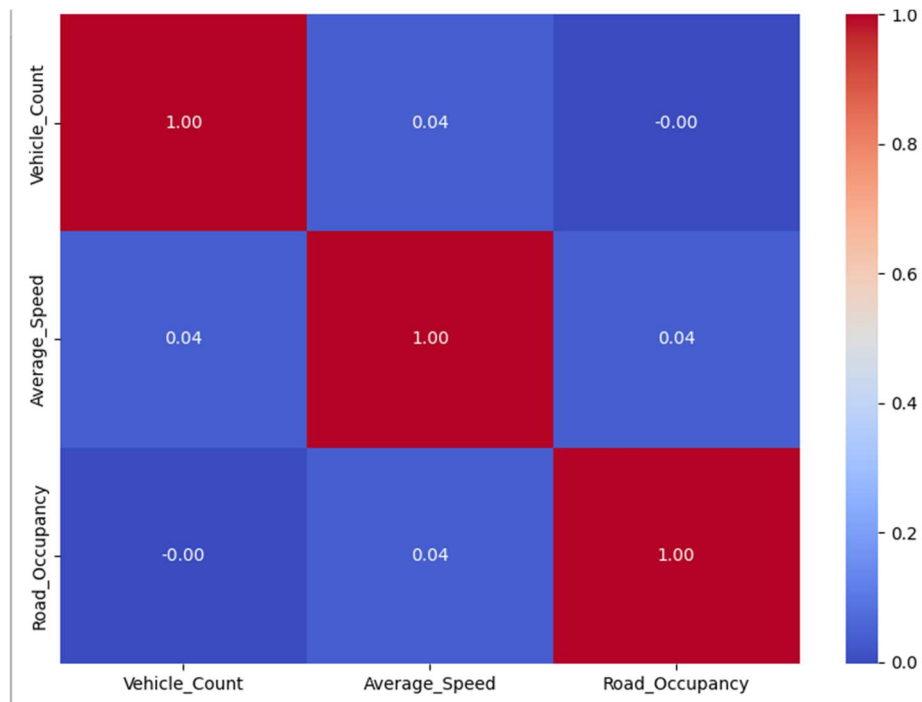


FIG NO: 05-CORRELATION HEATMAP OF TRAFFIC PARAMETERS

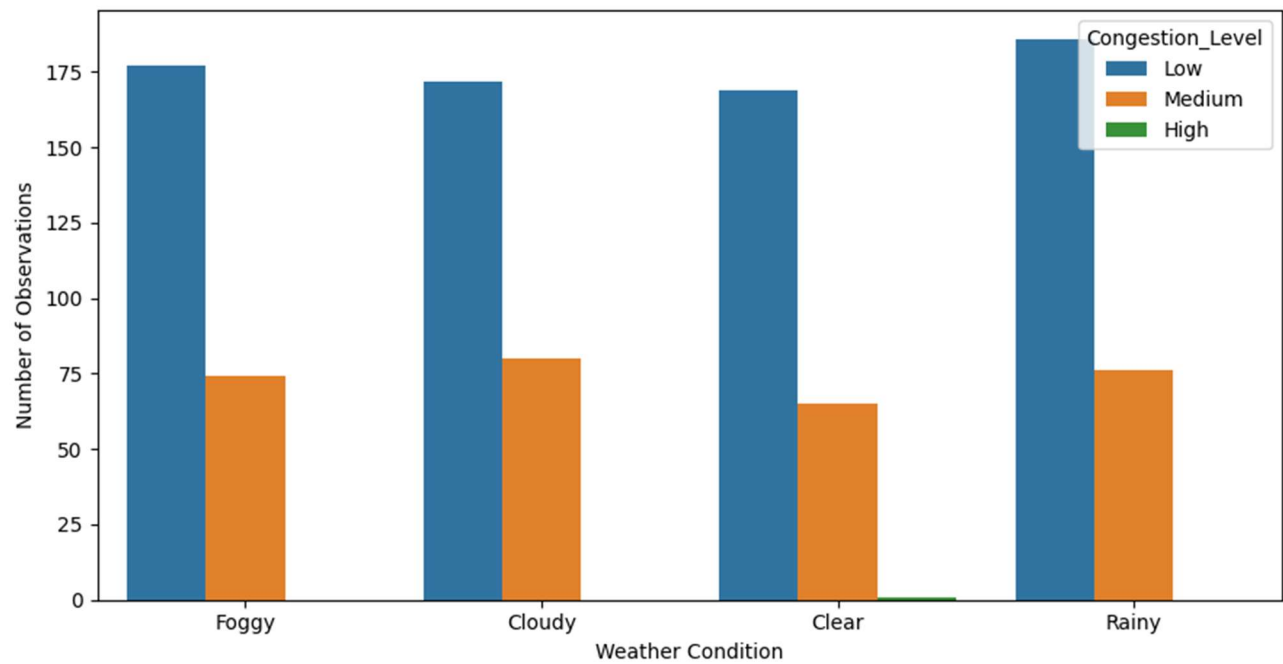


FIG N0:06-WEATHER CONDITION VS TRAFFIC CONGESTION

10. Results and Validation

First five rows:

	Vehicle_Count	Average_Speed	Time_of_Day	Day_of_Week	Weather_Condition	Road_Occupancy	Congestion_Level
0	122	72.618656	Evening	Thursday	Foggy	0.638610	Low
1	290	65.989868	Evening	Sunday	Cloudy	0.387718	Low
2	126	39.764945	Afternoon	Sunday	Cloudy	0.804487	Low
3	91	11.572852	Morning	Wednesday	Foggy	0.138341	Low
4	208	28.807415	Afternoon	Tuesday	Foggy	0.608792	Medium

Dataset shape:

(1000, 7)

Column names:

Index(['Vehicle_Count', 'Average_Speed', 'Time_of_Day', 'Day_of_Week',
'Weather_Condition', 'Road_Occupancy', 'Congestion_Level'],

Data types:

Vehicle_Count	int32
---------------	-------

Average_Speed	float64
---------------	---------

Time_of_Day	object
-------------	--------

Day_of_Week	object
-------------	--------

Weather_Condition	object
-------------------	--------

Road_Occupancy	float64
----------------	---------

Congestion_Level	object
------------------	--------

dtype: object

Missing values:

Vehicle_Count	0
---------------	---

Average_Speed	0
---------------	---

Time_of_Day	0
-------------	---

Day_of_Week	0
-------------	---

Weather_Condition	0
Road_Occupancy	0
Congestion_Level	0

dtype: int64

Duplicate rows:

0

Statistical summary:								
	Vehicle_Count	Average_Speed	Time_of_Day	Day_of_Week	Weather_Condition	Road_Occupancy	Congestion_Level	
count	1000.000000	1000.000000	1000	1000	1000	1000.000000	1000	
unique	NaN	NaN	4	7	4	NaN	3	
top	NaN	NaN	Morning	Sunday	Rainy	NaN	Low	
freq	NaN	NaN	260	168	262	NaN	704	
mean	159.911000	45.410078	NaN	NaN	NaN	0.521460	NaN	
std	79.781452	20.524386	NaN	NaN	NaN	0.274845	NaN	
min	20.000000	10.225278	NaN	NaN	NaN	0.050011	NaN	
25%	91.750000	26.822019	NaN	NaN	NaN	0.296359	NaN	
50%	162.500000	46.329212	NaN	NaN	NaN	0.518996	NaN	
75%	231.250000	63.113890	NaN	NaN	NaN	0.764294	NaN	
max	299.000000	79.958961	NaN	NaN	NaN	0.998430	NaN	

Dataset after preprocessing:								
	Vehicle_Count	Average_Speed	Time_of_Day	Day_of_Week	Weather_Condition	Road_Occupancy	Congestion_Level	
0	122	72.618656	Evening	Thursday	Foggy	0.638610	Low	
1	290	65.989868	Evening	Sunday	Cloudy	0.387718	Low	
2	126	39.764945	Afternoon	Sunday	Cloudy	0.804487	Low	
3	91	11.572852	Morning	Wednesday	Foggy	0.138341	Low	
4	208	28.807415	Afternoon	Tuesday	Foggy	0.608792	Medium	

Mean values:

Vehicle_Count	159.911000
Average_Speed	45.410078
Road_Occupancy	0.521460

dtype: float64

Median values:

Vehicle_Count	162.500000
Average_Speed	46.329212

Road_Occupancy	0.518996
----------------	----------

dtype: float64

Standard deviation:

Vehicle_Count	79.781452
Average_Speed	20.524386
Road_Occupancy	0.274845

dtype: float64

Congestion class distribution:

Congestion_Level

Low 704

Medium 295

High 1

Name: count, dtype: int64

Congestion_Level			
High	273.000000	14.665643	0.953858
Low	132.809659	49.460740	0.446778
Medium	224.203390	35.847632	0.698220

11. Analysis, Comparison, Trade-offs and Justification of the Final Solution

The Decision Tree is selected because it is suitable for classification and can represent learned decisions as interpretable rules. This is useful in traffic management because decision conditions can be inspected rather than treating the model as a completely opaque predictor. Feature-importance values also provide a direct way to examine which traffic parameters contribute to the model.

Trade-offs

Aspect	Advantage	Trade-off / Limitation
Interpretability	Decision rules can be visualized and explained.	A very deep tree can become difficult to interpret.
Training	Generally straightforward and fast for tabular data.	Performance depends on preprocessing and hyperparameter choices.
Non-linearity	Can model non-linear decision boundaries.	Uncontrolled growth can lead to overfitting.
Feature Analysis	Provides feature-importance values.	Importance does not necessarily imply causal influence.
Real-time Use	Prediction after training can be computationally light.	Input data must arrive reliably and be processed consistently.

Justification of the Final Solution

The proposed Decision Tree pipeline provides a clear end-to-end solution matching the assignment requirements: NumPy/Pandas-based preprocessing and analysis, visualization, supervised classification, feature-importance analysis, evaluation, and prediction on new traffic observations. The final model configuration and its performance should be justified using the actual experimental results obtained from execution.

12. Broader Considerations / SDG Relevance

The assignment explicitly maps the project to SDG 9 – Industry, Innovation and Infrastructure and SDG 11 – Sustainable Cities and Communities. The deliverables also identify SDG 13 – Climate Action as a broader consideration. A traffic congestion prediction system can contribute to these goals by supporting data-driven transportation management and more efficient use of urban infrastructure.

SDG 9 – Industry, Innovation and Infrastructure

The system demonstrates the use of machine learning and data-processing technologies in intelligent transportation infrastructure. Predictive analytics can support more responsive management of existing road infrastructure.

SDG 11 – Sustainable Cities and Communities

Accurate congestion information can support route management, traffic signal timing, and congestion-reduction strategies, contributing to more efficient and responsive urban mobility.

SDG 13 – Climate Action

Reducing unnecessary congestion and stop-and-go traffic has the potential to reduce fuel wastage and emissions. The magnitude of such benefits should be established using real deployment data rather than assumed performance.

Ethical, Bias and Privacy Considerations

- Sensor and camera data should be collected and processed in accordance with applicable privacy and data-protection requirements.
- Training data should represent different times, days, weather conditions, and traffic situations to reduce systematic bias.
- Predictions should support, rather than blindly replace, responsible traffic-management decisions.
- Model uncertainty and unusual traffic conditions should be considered before taking high-impact actions.
- The system should be monitored after deployment because traffic patterns can change over time.

13. Conclusion, Limitations and Possible Improvements

Conclusion

The proposed project develops an intelligent traffic congestion classification pipeline using NumPy, Pandas, and a Decision Tree classifier. It integrates data preprocessing, statistical analysis, visualization, feature importance, model evaluation, and prediction on new traffic data. The completed experimental results should be used to determine the effectiveness of the final model and to justify its suitability for the intended traffic-management application.

Limitations

- Model quality is dependent on the size, quality, and representativeness of the traffic dataset.
- A single Decision Tree may overfit when its complexity is not controlled.
- Traffic conditions can change rapidly and may include events or incidents absent from the training data.
- Sensor errors, missing observations, or inconsistent measurement intervals can affect prediction reliability.
- Feature importance indicates model contribution, not necessarily real-world causation.

Possible Improvements

- Collect larger and more diverse real-time datasets from multiple road segments.
- Perform systematic hyperparameter tuning and cross-validation.
- Compare the Decision Tree with Random Forest, Gradient Boosting, or other suitable classifiers.
- Introduce additional features such as incidents, road works, holidays, special events, and historical congestion.
- Develop a live dashboard or API for real-time prediction.
- Monitor model drift and periodically retrain the system with recent traffic data.
- Evaluate latency, robustness, and fairness under real deployment conditions.

14. Individual Contribution of Group Members

Complete the following table according to the actual work performed by each group member.

Group Member	Assigned Work	Contribution / Work Completed
THIRUVENKADAPRASATH.S	[Dataset & preprocessing]	DONE
KAMESHWARAN.K	[EDA & visualization]	DONE
ANIRUDHAN.S	[Report, testing & validation]	DONE

15. References

Replace or supplement the entries below with the exact sources actually used in the implementation.

1. Python Software Foundation. Python Documentation.
2. NumPy Documentation.
3. Pandas Documentation.
4. scikit-learn Documentation
5. Matplotlib Documentation.
6. Seaborn Documentation.
7. Dataset source used for the project

Appendix A – Output Insertion Checklist

- Dataset/sample preview screenshot
- Missing-value / preprocessing output screenshot
- Statistical summary output
- At least five meaningful traffic visualizations
- Decision Tree visualization
- Feature-importance graph/table
- Confusion matrix
- Accuracy, precision, recall and F1-score
- New/unseen traffic prediction output
- Any additional execution screenshots required by the faculty